

Scenario-01

"Test the search feature."

What's Missing in This Weak Prompt?

1. Instruction

Instruction to search is missing

1. Context

- What application/product?
- E-commerce search?

2. User Persona / Role

Who uses it?

- Customer
- Seller
- Admin

3. Example:

- Search example is missing

4. Out Format

Expected output format is missing

5. Tone

Tone is missing (Not much imported for search)

Week Prompt output

TCID	Description	Priority	Preconditions	Steps	Expected Result
TC_011	Verify search returns relevant results for valid keyword	High	User is logged into application and searchable data exists	1. Navigate to search bar 2. Enter valid keyword 3. Click Search	Relevant search results should be displayed accurately
TC_012	Verify search with partial keyword	Medium	Searchable data exists	1. Enter partial keyword in search field 2. Click Search	Matching partial search results should be displayed
TC_013	Verify search with invalid keyword	Medium	User is on search page	1. Enter invalid/non-existing keyword 2. Click Search	System should display "No Results Found" message
TC_014	Verify search functionality with special characters	High	Search feature is enabled	1. Enter special characters (@,#,\$,%) 2. Click Search	System should handle input securely without crash or SQL injection

TCID	Description	Priority	Preconditions	Steps	Expected Result
TC_015	Verify search performance with large dataset	High	Large volume of searchable records available	1. Enter common keyword 2. Trigger search	Search results should load within acceptable response time
TC_016	Verify search is case insensitive	Medium	Searchable records exist	1. Search using uppercase keyword 2. Search using lowercase keyword	Results should remain consistent regardless of case
TC_017	Verify search with empty input field	Medium	User is on search page	1. Leave search field blank 2. Click Search	Validation message should prompt user to enter keyword
TC_018	Verify search result pagination	Medium	Multiple search results available	1. Search common keyword 2. Navigate through pages	Pagination should work correctly without missing data
TC_019	Verify usability of search feature	Low	Search functionality available	1. Observe search bar visibility and placeholder text 2. Perform search	Search feature should be user-friendly and intuitive
TC_020	Verify security against SQL injection in search field	High	Application connected to database	1. Enter SQL injection payload (' OR '1'='1') 2. Click Search	Application should sanitize input and prevent unauthorized access

Rewritten ICEPOT prompt

Context

Act as a Senior Test Engineer. Ecommerce Domain and application

Instruction

[MANDATORY] Test the search feature of amazon Ecommerce portal

[MANDATORY] Generate only 10 testcases

[MANDATORY] Include Functional and Non functional

[MANDATORY] Include Positive and negative cases

Example:

TCID | Description | Priority | Precomditions | Steps | Expected Result

Persona

Act as a Test lead to review the testcases

Output

Downloadable excel

Tone:

Professional Tone with clarity

Rewritten ICEPOT prompt output

I can't directly generate and attach an actual Excel (.xlsx) file in this chat. However, the content below is Excel-ready—copy it into Excel and save as .xlsx.

TCID	Description	Type	Priority	Preconditions	Steps	Expected Result
TC_001	Verify search with a valid product keyword	Functional, Positive	High	User is on Amazon home page	1. Enter "iPhone 15" in search box. 2. Click Search.	Relevant products matching "iPhone 15" are displayed.
TC_002	Verify search using partial product name	Functional, Positive	High	User is on Amazon home page	1. Enter partial keyword "iPh". 2. Click Search.	Search results display products related to the partial keyword.
TC_003	Verify search using category filter	Functional, Positive	Medium	User is on Amazon home page	1. Select Electronics category. 2. Search for "Headphones".	Results displayed belong primarily to the selected category.
TC_004	Verify search with special characters only	Functional, Negative	Medium	User is on Amazon home page	1. Enter "@#\$%^&*" in search box. 2. Click Search.	Appropriate message or no relevant results are displayed without application error.

TCID	Description	Type	Priority	Preconditions	Steps	Expected Result
TC_005	Verify search with non-existing product	Functional, Negative	High	User is on Amazon home page	1. Enter random text "XYZ123INVALIDPRODUCT". 2. Click Search.	No matching products found message is displayed.
TC_006	Verify search with empty search field	Functional, Negative	High	User is on Amazon home page	1. Leave search field blank. 2. Click Search.	System displays validation or default behavior without errors.
TC_007	Verify search suggestions/autocomplete	Functional, Positive	Medium	User is on Amazon home page	1. Type "Laptop" slowly in search box.	Relevant autocomplete suggestions appear dynamically.
TC_008	Verify search performance for common keyword	Non-Functional, Positive	High	Stable network connection	1. Search for "Mobile". 2. Measure response time.	Search results load within acceptable SLA (e.g., ≤3 seconds).
TC_009	Verify search under high user load	Non-Functional, Positive	High	Load testing environment available	1. Simulate concurrent users performing searches. 2. Monitor system behavior.	Search service remains stable and meets

TCID	Description	Type	Priority	Preconditions	Steps	Expected Result
TC_010	Verify search security against script injection	Functional/Security, Negative	High	User is on Amazon home page	1. Enter <code><script>alert('test')</script></code> in search box. 2. Click Search.	performance benchmarks.
						Input is sanitized; script is not executed and application remains secure.

Test Lead Review Summary

-  Functional test coverage included.
-  Non-functional test coverage included (Performance & Load).
-  Positive scenarios included.
-  Negative scenarios included.
-  High-priority business-critical search scenarios covered.
-  Security validation included.

Comparison

1. Data returned is more relevant
2. It covers all the expected scenarios in ICEPOT method
3. More clarity

Scenario – 02

"Generate test cases for file upload."

What's Missing in This Weak Prompt?

1. Test cases depend on the application type and business rules.
2. Different upload features have different validations (Profile picture upload, Product image upload, Invoice upload, Bulk CSV upload)
3. Defines coverage expectations. (Include Functional and Non-Functional cases).
4. Ensures consistent deliverables(TCID | Description | Priority | Preconditions | Steps | Expected Result)

Week Prompt output

If you're generating test cases for a file upload feature, it's useful to cover **functional, validation, security, performance, and usability** scenarios.

Functional Test Cases

TC ID	Test Case	Expected Result
FU-001	Upload a valid file with supported format	File uploads successfully
FU-002	Upload multiple valid files (if supported)	All files upload successfully
FU-003	Upload a file using drag-and-drop	File uploads successfully
FU-004	Upload a file using file browser	File uploads successfully
FU-005	Cancel upload before completion	Upload stops and file is not saved
FU-006	Replace an already selected file	New file replaces old one
FU-007	Remove selected file before upload	File is removed from upload queue
FU-008	Retry upload after failure	Upload completes successfully

File Type Validation

TC ID	Test Case	Expected Result
FU-009	Upload supported file type (e.g., PDF)	Upload succeeds
FU-010	Upload unsupported file type (.exe)	Error message displayed
FU-011	Upload file with uppercase extension (.PDF)	Upload succeeds if type is supported

TC ID	Test Case	Expected Result
FU-012	Upload file with no extension	Validation occurs according to requirements
FU-013	Upload renamed file with incorrect extension	System validates actual file type
FU-014	Upload corrupted file	Appropriate error displayed

File Size Validation

TC ID	Test Case	Expected Result
FU-015	Upload file below size limit	Upload succeeds
FU-016	Upload file exactly at size limit	Upload succeeds
FU-017	Upload file exceeding size limit	Error message displayed
FU-018	Upload empty (0 KB) file	Handled according to requirements
FU-019	Upload extremely large file	Upload rejected or processed per requirements

Filename Validation

TC ID	Test Case	Expected Result
FU-020	Upload file with normal filename	Upload succeeds
FU-021	Upload file with special characters in name	Handled correctly
FU-022	Upload file with spaces in filename	Upload succeeds
FU-023	Upload file with very long filename	Validation works correctly
FU-024	Upload file with Unicode characters	Upload succeeds
FU-025	Upload duplicate filename	System handles duplicates correctly

Security Test Cases

TC ID	Test Case	Expected Result
FU-026	Upload executable file (.exe)	Blocked if not allowed
FU-027	Upload file containing malware	File detected/rejected
FU-028	Upload script file (.js, .php, .jsp)	Blocked according to policy

TC ID	Test Case	Expected Result
FU-029	Attempt path traversal in filename (../.././file.txt)	Upload rejected/sanitized
FU-030	Upload file with SQL injection characters in name	No security impact
FU-031	Upload file with XSS payload in filename	Filename safely encoded
FU-032	Upload file without authentication	Access denied

Performance Test Cases

TC ID	Test Case	Expected Result
FU-033	Upload large file under normal network conditions	Completes within acceptable time
FU-034	Upload multiple files simultaneously	System remains stable
FU-035	Upload during peak load	Performance remains acceptable
FU-036	Upload over slow network	Appropriate progress indication shown
FU-037	Resume upload after network interruption (if supported)	Upload resumes successfully

Usability & UI Test Cases

TC ID	Test Case	Expected Result
FU-038	Verify upload progress indicator	Progress displayed accurately
FU-039	Verify success message after upload	Clear confirmation displayed
FU-040	Verify error message for invalid upload	User-friendly error displayed
FU-041	Verify supported file formats are shown	Information visible to user
FU-042	Verify upload button disabled when no file selected	Button behaves correctly
FU-043	Verify accessibility using keyboard navigation	Upload functionality accessible

API/Backend Validation (if applicable)

TC ID	Test Case	Expected Result
FU-044	Verify file stored in correct location	File stored correctly

TC ID	Test Case	Expected Result
FU-045	Verify metadata saved in database	Metadata saved accurately
FU-046	Verify upload API returns success status	Correct response code returned
FU-047	Verify upload API handles invalid requests	Proper error response returned
FU-048	Verify file deletion after upload (if supported)	File removed successfully

Rewritten ICEPOT prompt

Context: Act as a Senior Test Engineer. write test case file upload functionality to banking application

Instruction:

- Include Functional and Non-Functional test cases.
- Include Positive and Negative scenarios.
- Cover file type validation, file size validation, performance, security, and usability.
- Generate exactly 10 test cases.

Example:

- TCID | Description |Priority|Precomditions| Steps |Expected Result

Persona

- Act as a Test lead and review the testcases and give feed back

Output

- Downloadable excel

Tone:

- Professional Tone with clarity

Rewritten ICEPOT prompt Output.

Below is a **Test Case File (Excel-ready format)** for **File Upload Functionality in a Banking Application**.

You can directly copy this into Excel using CSV import.

 **Test Case File (CSV Format)**

TCID	Description	Priority	Preconditions	Steps	Expected Result
TC_001	Verify successful upload of valid file type (PDF)	High	User is logged into banking application with upload permission	"1. Navigate to File Upload section 2. Click Browse 3. Select valid PDF file 4. Click Upload"	"File uploaded successfully and confirmation message displayed"
TC_002	Verify upload is rejected for invalid file type (EXE)	High	User is logged in and on upload page	"1. Click Browse 2. Select .exe file 3. Click Upload"	"System rejects file and displays 'Invalid file type' error message"
TC_003	Verify file upload with size exceeding maximum limit	High	User is logged in and upload limit defined (e.g., 10MB)	"1. Select file greater than 10MB 2. Click Upload"	"Upload fails with 'File size exceeds limit' error message"
TC_004	Verify upload of file at maximum allowed size boundary	Medium	User is logged in and max file size defined	"1. Select file exactly at max size limit 2. Click Upload"	"File uploaded successfully without errors"
TC_005	Verify multiple file upload restriction (if only single file allowed)	Medium	User is logged in and system allows only single file upload	"1. Try selecting multiple files 2. Attempt upload"	"System restricts multiple file selection and shows validation message"
TC_006	Verify malware/virus scanning during file upload	High	User is logged in and antivirus scanning enabled	"1. Upload file containing malware signature 2. Submit upload"	"System blocks file upload and flags security threat"
TC_007	Verify system performance during large file upload	High	User is logged in and performance testing environment is set	"1. Upload large but valid file 2. Measure response time"	"File uploaded within acceptable SLA time (e.g., <5 seconds) or performance threshold met"
TC_008	Verify upload behavior during network interruption	High	User is logged in and upload in progress	"1. Start uploading file 2. Disconnect network mid-upload"	"Upload fails gracefully with appropriate error message and no corrupted file stored"
TC_009	Verify handling of special characters in file name	Medium	User is logged in	"1. Select file with special characters (!,@,#,\$,%) in name 2. Upload file"	"System sanitizes file name and uploads successfully or rejects with proper validation message"
TC_010	Verify usability of drag and drop functionality	Low/Medium	User is logged in and UI supports drag-drop	"1. Drag file into upload area 2. Drop file"	"File is accepted and uploaded successfully with visual confirmation"

Test Lead Review & Feedback

Strengths

- Good coverage of **functional and non-functional scenarios**
- Includes **positive and negative test cases**
- Security aspects like malware scanning and invalid file types are covered
- Performance and usability scenarios are included
- Boundary condition testing is addressed (file size limit)

Improvements Required

1. **Missing API-level validation tests**
 - No coverage for backend validation or direct API upload bypass attempts.
2. **Error message validation is not detailed**
 - Expected results should include exact error codes/messages where applicable.
3. **Audit & Logging not covered**
 - Banking applications require verification of upload logs for compliance.
4. **Encryption & Data Security**
 - No test case verifying file encryption during upload or storage.

5. **Concurrency testing missing**
 - Multiple users uploading simultaneously is not validated.
6. **Session timeout scenario missing**
 - Upload during session expiry should be tested.
7. **File integrity validation**
 - No checksum/hash verification after upload.

Comparison

1. Data returned is more relevant
2. It covers all the expected scenarios in ICEPOT method
3. More clarity

Scenario-03

“Test the cart functionality.”

What’s Missing in This Weak Prompt?

1. It doesn’t specify Web app, mobile app, or API, Guest cart or logged-in user cart
2. Missing details like Can users add out-of-stock items?
3. It doesn’t clarify Add/remove/update only?
4. Missing Performance expectations, Security requirements, API vs UI testing scope
5. It doesn’t specify Manual vs automation test cases

Week prompt output

Here are comprehensive test cases for shopping cart functionality, covering functional, validation, edge, UI, and backend scenarios.

1. Add to Cart Test Cases

TC ID	Test Case	Expected Result
CART-001	Add a single valid product to cart	Product is added successfully
CART-002	Add multiple different products	All products appear in cart
CART-003	Add same product multiple times	Quantity increases or duplicates handled per design
CART-004	Add product with variants (size/color)	Correct variant is added
CART-005	Add out-of-stock product	Proper error shown, not added
CART-006	Add product from product detail page	Product added successfully
CART-007	Add product from listing page	Product added successfully

2. Update Cart Quantity

TC ID	Test Case	Expected Result
CART-008	Increase product quantity	Quantity updates correctly
CART-009	Decrease product quantity	Quantity updates correctly
CART-010	Set quantity to zero	Item removed from cart
CART-011	Enter negative quantity	Validation error shown
CART-012	Enter extremely large quantity	System enforces max limit
CART-013	Update quantity beyond stock	Error or max allowed enforced

3. Remove Items from Cart

TC ID	Test Case	Expected Result
CART-014	Remove single item	Item removed successfully
CART-015	Remove all items one by one	Cart becomes empty

TC ID	Test Case	Expected Result
CART-016	Clear entire cart	All items removed
CART-017	Remove item after page refresh	State remains consistent

4. Price & Calculation Validation

TC ID	Test Case	Expected Result
CART-018	Verify subtotal calculation	Correct sum of items
CART-019	Verify tax calculation	Tax applied correctly
CART-020	Verify discount/coupon applied	Discount reflected properly
CART-021	Verify shipping cost	Correct shipping applied
CART-022	Verify total amount	Final total is accurate
CART-023	Price change in product after adding to cart	Cart reflects updated rules (or old price as per requirement)

5. Coupon & Promotions

TC ID	Test Case	Expected Result
CART-024	Apply valid coupon	Discount applied
CART-025	Apply invalid coupon	Error message shown
CART-026	Apply expired coupon	Coupon rejected
CART-027	Apply coupon twice	Not allowed
CART-028	Remove applied coupon	Discount removed

6. Cart Persistence & Session

TC ID	Test Case	Expected Result
CART-029	Add items and refresh page	Cart persists
CART-030	Logout and login again	Cart retained (if designed)
CART-031	Add item in one device, open another device	Sync behavior verified
CART-032	Session expiry	Cart behavior as per requirement

7. Stock & Availability Handling

TC ID	Test Case	Expected Result
CART-033	Item goes out of stock after adding	User notified
CART-034	Partial stock availability	Quantity adjusted or error shown
CART-035	Stock updated while in cart	Cart reflects correct stock

8. UI/UX Test Cases

TC ID	Test Case	Expected Result
CART-036	Cart icon shows correct item count	Count matches cart

TC ID	Test Case	Expected Result
CART-037	Empty cart UI	Proper “cart is empty” message shown
CART-038	Price updates reflect instantly	UI updates without refresh
CART-039	Remove animation/feedback	Smooth UX behavior
CART-040	Mobile responsiveness	Cart works on all screen sizes

9. Performance & Load

TC ID	Test Case	Expected Result
CART-041	Add/remove items rapidly	No system crash
CART-042	Large number of cart items	UI and backend handle efficiently
CART-043	Concurrent updates from multiple tabs	Consistent state maintained

10. Security Test Cases

TC ID	Test Case	Expected Result
CART-044	Manipulate price in frontend	Backend validates correct price
CART-045	Tamper cart API requests	Unauthorized changes blocked
CART-046	Add restricted product via API	Request rejected
CART-047	Coupon abuse attempts	Proper validation enforced

Rewritten ICEPOT prompt

Context: Act as Senior Test Engineer. Filpkart web application writes test case for add to cart functionality.

Instruction:

- Verify Web application adds cart functionality
- Add only in stock products
- Remove already in cart product
- Include Functional and Non-Functional test cases.
- Include Positive and Negative scenarios.
- Cover file type validation, file size validation, performance, security, and usability.
- Generate exactly 10 test cases.

Example:

- TCID | Description |Priority|Precomditions| Steps |Expected Result

Persona

- Act as a Test lead and review the testcases

Output

- Downloadable excel

Tone:

- Professional Tone with clarity

Cart Functionality.

You can directly copy this into Excel (Data → From Text/CSV).

Test Case File (CSV Format)

TCID,Description,Priority,Preconditions,Steps,Expected Result

TC_001,Verify user can add in-stock product to cart,High,User is logged in and product is in stock,"1. Open product page 2. Click Add to Cart","Product is successfully added and cart count is updated"

TC_002,Verify user cannot add out-of-stock product to cart,High,User is logged in and product is marked out of stock,"1. Open out-of-stock product page 2. Click Add to Cart","System prevents add to cart and shows 'Out of stock' message"

TC_003,Verify adding multiple quantities of same in-stock product,Medium,User is logged in and product allows multiple quantity,"1. Add product to cart 2. Increase quantity 3. Update cart","Cart updates quantity correctly and total price is recalculated"

TC_004,Verify user can remove product from cart,High,User has at least one product in cart,"1. Go to cart page 2. Click Remove button","Product is removed and cart is updated successfully"

TC_005,Verify removing a product not present in cart,Medium,User is logged in with empty or different cart state,"1. Attempt to remove product via direct URL/API or stale session action","System shows error or ignores request without crash"

TC_006,Verify cart persistence after logout and login,High,User is logged in with items in cart,"1. Add product to cart 2. Logout 3. Login again","Previously added items remain in cart"

TC_007,Verify system performance while adding product to cart,High,User is logged in and product page is accessible,"1. Click Add to Cart 2. Measure response time","Product is added within acceptable SLA (e.g., <2–3 seconds)"

TC_008,Verify security against unauthorized cart manipulation,High,User is logged in and network tools enabled,"1. Modify request payload (product ID/SKU) 2. Send request","System rejects tampered request and prevents unauthorized cart update"

TC_009,Verify usability and UI feedback on add to cart action,Medium,User is logged in and product page is open,"1. Click Add to Cart 2. Observe UI changes","Button changes state, cart badge updates, and success message is displayed"

TC_010,Verify system handles invalid/tampered product identifier,High,User is logged in,"1. Manipulate product ID in request 2. Attempt add to cart","System validates product ID and blocks invalid or manipulated requests"

Test Lead Review & Feedback

Strengths

- Strong coverage of core Add to Cart workflows
- Includes positive and negative scenarios
- Good inclusion of:
 - Out-of-stock validation
 - Cart persistence
 - Remove functionality
 - Security against request tampering
- Performance and usability are also covered

Improvements Required

1. Missing API-level test separation

- **UI and backend validation should be split for better traceability**
- 2. Concurrency scenarios missing**
 - **Multiple users adding same limited-stock product simultaneously not tested**
- 3. Edge cases not fully covered**
 - **Session timeout during add-to-cart operation**
 - **Network retry / duplicate request handling**
- 4. Business rule validation missing**
 - **Coupon application impact on cart not included**
 - **Minimum/maximum quantity rules not validated**
- 5. Error message validation not detailed**
 - **Expected error codes/messages should be standardized**

Comparison

1. Data returned is more relevant
2. It covers all the expected scenarios in ICEPOT method
3. More clarity